Análise Completa — Menu Play (Sistema de Cardápio Digital)

> Documento de análise técnica do projeto **Menu Play**, gerado para orientar a retomada do desenvolvimento e facilitar a integração de novas pessoas no time.

---

1. Visão Geral do Projeto

O **Menu Play** é um SaaS de **cardápio digital** (estilo iFood/QR Code menu) com dois perfis de usuário:

- **Gerenciador (Dono do estabelecimento):** Cadastra empresa, categorias, produtos, vê pedidos e relatórios.
- **Cliente final:** Acessa o cardápio via subdomínio/QR Code, navega por categorias, adiciona itens ao carrinho e faz pedidos.

O sistema usa o modelo de **assinatura** (Mensal/Semestral/Anual) — o dono paga para ter seu cardápio na plataforma.

Status atual

~70% concluído. O fluxo de CRUD de categorias/produtos e o carrinho funcionam, mas **pedidos, relatórios, dashboard e checkout estão incompletos**.

---

2. Stack Tecnológica

Camada	Tecnologia
Framework	React 17 + Create React App 4
Roteamento	react-router-dom 5
UI	Material-UI v4 + styled-components + Bootstrap (landing page)
Backend/BaaS	Parse Server (hosted em back4app.com)
Gráficos	recharts
Máscaras	react-input-mask
CEP	cep-promise
Ícones	react-icons, @material-ui/icons

> **Atenção:** O `package.json` lista `next` como dependência, mas o projeto **não usa Next.js** — é CRA puro. Provavelmente um resquício de experimentação.

---

3. Arquitetura de Pastas

```
src/
├── App.js                    # Só renderiza <Routes />
├── index.js                  # Entry point CRA
├── routes.js                 # Toda a definição de rotas + init do Parse
├── services/
│   ├── auth.js               # PrivateRoute (protege rotas admin)
│   └── environments.js       # Chaves do Parse/back4app (hardcoded!)
├── context/                  # Sistema de carrinho (useReducer) — NÃO
USADO
│   ├── Types.js
│   └── cart/ (CartContext, CartReducer, CartState)
├── hooks/
│   └── useCart.jsx           # Carrinho real em uso (useState +
localStorage)
├── util/format.js            # formatPrice (Intl BRL)
├── components/               # Componentes reutilizáveis
│   ├── Menu/                 # Sidebar admin (AppBar + Drawer)
│   ├── MenuClient/           # Sidebar cliente
│   ├── MenuList/             # Itens dos menus (admin e cliente)
│   ├── CardProdutos/         # Grid de produtos na aba do menu
│   ├── InputMasks/           # Inputs CEP/CPF/CNPJ/Phone/Price/Hour
│   ├── Header, Nav, Footer, FooterGer, Button, Title, Logout
│   ├── Chart/                # Gráfico de linha (dados mockados)
│   ├── Pedidos/, Pendents/   # Componentes com dados MOCKADOS
│   └── Planos/               # Cards de planos de assinatura
└── pages/
    ├── Home/                 # Landing page (Bootstrap)
    ├── Login/                # Login via Parse.User.logIn
    ├── Register/             # Cadastro de usuário (Parse.User.signUp)
    ├── Checkout/             # Checkout de assinatura (Stepper)
    ├── admin/                # Área do gerenciador
    │   ├── Dashboard/        # VAZIO (placeholders)
    │   ├── Categoria/ + AddCategoria + EditCategoria
    │   ├── Produto/ + AddProduto + EditProduto
    │   ├── PedidosGer/       # Tabela sem dados (loadPedidos comentado)
    │   ├── Relatorios/       # Tabela sem dados
    │   └── Profile/          # Edição dados da empresa (completo)
    └── client/               # Área do cliente
        ├── Menu/             # Cardápio por subdomínio com abas
        ├── Menu/DetailProduto/   # Detalhe do produto + add ao carrinho
        ├── Menu/DetailEmpresa/   # Info da empresa
        ├── Cart/             # Carrinho (funcional)
        ├── Cart/NewPedido/   # Form de pedido (NÃO SALVA)
        └── Pedidos/          # Lista de pedidos (MOCKADA)
```

---

4. Modelo de Dados (Back4App/Parse)

Classes implícitas detectadas no código:

```
User (Parse.User)
├── username, password, email, nome

Empresa
├── id_user     → Pointer<User>
├── subdominio  → String (identifica a empresa na URL do cliente)
├── logo        → Parse.File
├── nome, cnpj, celular, telefone, email
├── inicioSemanaAberto, fimSemanaAberto
├── inicioHorarioNormal, fimHorarioNormal
├── inicioHorarioFeriado, fimHorarioFeriado
├── cep, uf, cidade, logradouro, bairro, numero

Categoria
├── id_empresa  → Pointer<Empresa>
├── nome
├── imagem      → Parse.File

Produto
├── id_categoria → Pointer<Categoria>
├── nome, descricao, preco
├── imagem        → Parse.File

Pedido  ⚠️ NÃO CRIADO no código atual (fluxo de save incompleto)
├── (estrutura presumida: nome, cpf, tipoPagamento, tipoEntrega,
│    valorTotal, status, createdAt)
```

---

5. Mapeamento de Rotas

Rota	Tipo	Função
`/`	Pública	Landing page (Home)
`/login`	Pública	Login do gerenciador
`/register`	Pública	Cadastro simples
`/checkout`	Pública	Checkout de assinatura (Stepper 3 passos)
`/dashboard`	Privada	Painel admin (vazio)
`/categoria`	Privada	Lista categorias
`/categoria/new`	Privada	Nova categoria
`/categoria/edit/:idCategoria`	Privada	Editar categoria

Rota	Tipo	Função
`/categoria/:idCategoria/produto/`	Privada	Produtos da categoria
`/categoria/:idCategoria/produto/new`	Privada	Novo produto
`/categoria/:idCategoria/produto/edit/:idProduto`	Privada	Editar produto
`/pedidos/admin`	Privada	Pedidos recebidos (sem dados)
`/relatorios`	Privada	Relatórios (sem dados)
`/profile`	Privada	Dados da empresa
`/:subdominio/menu`	Pública + CartProvider	Cardápio do cliente
`/:subdominio/menu/item/:id`	Pública + CartProvider	Detalhe do produto
`/:subdominio/empresa`	Pública + CartProvider	Detalhe da empresa
`/:subdominio/cart`	Pública + CartProvider	Carrinho
`/:subdominio/newpedido`	Pública + CartProvider	Form de novo pedido
`/:subdominio/pedidos`	Pública + CartProvider	Pedidos do cliente

6. Fluxos Principais

6.1 Fluxo do Gerenciador (Admin)

1. Na landing page clica em **Login** → `/login`
2. Autentica via `Parse.User.logIn` → redireciona `/dashboard`
3. `PrivateRoute` (`src/services/auth.js:5`) verifica `Parse.User.current().authenticated()`
4. No `/categoria` cadastra categorias com upload de imagem (Parse.File)
5. Em cada categoria cadastra produtos (nome, descrição, preço, imagem)
6. Em `/profile` preenche dados da empresa + CEP (via `cep-promise`)

6.2 Fluxo do Cliente

1. Acessa `/{subdominio}/menu` (ex: `/pizzaria-do-joao/menu`)
2. Sistema busca `Empresa` por `subdominio` (`src/pages/client/Menu/index.jsx:34`)
3. Lista categorias em **abas** (Material-UI `TabContext`)
4. `CardProdutos` busca produtos da categoria e renderiza cards

5. Clica num produto → `/:subdominio/menu/item/:id` (DetailProduto)
6. Adiciona ao carrinho via `useCart().addProduct()` (salva em localStorage `@MenuPlay:cart`)
7. No carrinho (`/:subdominio/cart`) ajusta quantidades e remove itens
8. "Fazer pedido" deveria levar ao `/newpedido` → **mas o botão não tem `onClick` nem `Link` ⚠️**

6.3 Fluxo de Assinatura (Checkout)

- `/checkout` tem 3 steps: UserForm → PaymentForm → Order
- **PROBLEMA:** os dados do UserForm não são passados para o `handleSignUp` do checkout (usa `let data = "teste"` na linha 52) — cadastra user com dados vazios

---

7. Estado do Carrinho — ⚠️ Dupla Implementação

Existe **redundância** no gerenciamento de carrinho:

Arquivo	Status
`src/context/cart/` (CartContext + CartReducer + CartState)	**Não usado** — morto
`src/hooks/useCart.jsx`	**Em uso** — useState + localStorage

O `routes.js` importa o `CartProvider` de `useCart.jsx` (linha 30), não o `CartState`. A pasta `src/context/cart/` pode ser removida.

---

8. O que está COMPLETO e funcional ✅

- **Landing page** (`Home` + `Planos` + `Footer`)
- **Login e Logout** (Parse.User)
- **CRUD de Categorias** (listar, criar, editar, excluir)
- **CRUD de Produtos** (listar, criar, editar, excluir)
- **Upload de imagens** (Parse.File)
- **Profile da empresa** (com máscaras + busca CEP)
- **Menu do cliente** (abas + cards + subdomínio)
- **Detalhe do produto** + adicionar ao carrinho
- **Carrinho** (adicionar/remover/incrementar/decrementar + total)
- **Máscaras de input** (CPF, CNPJ, CEP, phone, hora)

9. O que está INCOMPLETO ou com BUG ✕

Funcionalidade	Problema
Dashboard	`src/pages/admin/Dashboard/index.jsx` — papers vazios, `Chart`/`Pedidos`/`Pendents` não importados
PedidosGer	`src/pages/admin/PedidosGer/index.jsx:28-36` — `loadPedidos` comentado; `handleUpdateStatus` vazio
Relatorios	`loadPedidos` vazio; bug em `:95` compara objeto literal `{relatorio}` em vez de string; `toLocateString` (typo → `toLocaleString`)

Funcionalidade	Problema
Pedidos (cliente)	`src/pages/client/Pedidos/index.jsx` — dados hardcoded (2 Accordions estáticos)
NewPedido	`handleSubmit` vazio (não salva no Parse); inputs `nome/cpf/celular` sem `value`/`onChange` amarrados
Cart → Fazer pedido	`src/pages/client/Cart/index.jsx:139` — botão sem `onClick` ou `Link`
Checkout	`handleSignUp` cria user com `data="teste"`; UserForm não comunica dados ao pai
Register	`alert("User created!")` em inglês; sem redirect após cadastro
Login → Senha	Link "Esqueceu a senha?" aponta para `"#"` (não implementado)
Gerar QR Code	Item no menu (`/qrcode`) mas **rota não existe** no `routes.js`
Logout no menu cliente	`MenuClient` não tem botão de logout (cliente não autentica — ok por design)
Auth guards cliente	Rotas cliente são públicas (correto, mas sem validação de subdomínio inexistente → crash)

10. Problemas de Segurança e Qualidade

1. **Chaves do Parse hardcoded** em `src/services/environments.js` — expostas no bundle. Mover para `.env` + `REACT_APP_*`.
2. **Sem tratamento de erro quando subdomínio não existe** — `objectEmpresa.get('logo')` crasha se `undefined` (`Menu/index.jsx:36`, `DetailEmpresa/index.jsx:28`).
3. **`useEffect` com dependência errada** em vários lugares (`[categorias]`, `[produtos]`, `[empresa]`) — pode causar loops infinitos. Ex: `Categoria/index.jsx:59`, `Produto/index.jsx:64`.
4. **`index.html:32`** tem `</script>` órfão (tag de fechamento sem abertura correspondente).
5. **Mix de Bootstrap (landing) + Material-UI (app)** — duplicação de CSS, pesos diferentes.
6. **`package.json` inclui `next` sem uso** + falta `prop-types` apesar de usado em `Title`.
7. **Sem testes** apesar de `@testing-library` nas deps.
8. **`eslint` usa `react-app`** mas há erros evidentes não capturados (ex: `data = {...}` em `UserForm` reatribui `const`).

11. Como rodar

```
yarn install        # ou npm install (cuidado: package-lock + yarn.lock presentes)
yarn start          # http://localhost:3000
yarn build          # produção → /build
yarn test           # sem testes escritos
```

Requer conta ativa no **back4app** com App ID `IGOO1ABZyrepdHex...` (provavelmente expirado após 5 anos).

---

12. Sugestão de Roadmap para Retomar

Prioridade alta (destravar o core)

1. Mover chaves do Parse para `.env` (`REACT_APP_PARSE_APP_ID`, etc.)
2. Implementar salvar **Pedido** no Parse em `NewPedido` + botão "Fazer pedido" no Cart
3. Implementar `loadPedidos` no `PedidosGer` (query Parse por `id_empresa`)
4. Corrigir bugs de digitação: `toLocateString` → `toLocaleString`; `{relatorio}` → `relatorio`

Prioridade média (fechar gaps)

5. Dashboard: conectar `Chart`/`Pedidos` com dados reais do Parse
6. Fluxo de checkout: passar dados do UserForm para o signUp
7. Adicionar rota `/qrcode` ou remover item do menu
8. Tratar subdomínio inexistente (tela 404 amigável)
9. Remover pasta `src/context/cart/` morta
10. Remover `next` do `package.json`

Prioridade baixa (modernização)

11. Atualizar React 17 → 18, Material-UI v4 → MUI v5, CRA → Vite
12. Trocar `react-router-dom` v5 → v6
13. Adicionar TypeScript
14. Centralizar acesso ao Parse num `services/api.js`

---

13. Resumo executivo

O **Menu Play** é um SaaS de cardápio digital (React + Parse/back4app) com duas frentes: um painel admin onde o dono do restaurante cadastra sua empresa, categorias e produtos; e uma vitrine pública acessada por subdomínio onde o cliente monta o carrinho e faz pedidos. **Cerca de 70% do CRUD (categorias/produtos/empresa) e o carrinho estão funcionando**, mas o **fechamento do pedido, listagem de pedidos, dashboard, relatórios e o checkout de assinatura estão incompletos** — faltam ligar ao backend, corrigir bugs de digitação e tratar erros. As chaves do Parse estão hardcoded (precisam ir para `.env`) e há código morto de carrinho (`src/context/cart/`) que deve ser removido. É um projeto viável de retomar: a base de dados Parse, o roteamento e os componentes UI já estão estruturados — o esforço principal está em **completar o fluxo de pedidos de ponta a ponta** (cliente cria → salva no Parse → admin vê → muda status).